

From great program to great product

A native rebuild of the Functional Bodybuilding apps, backend, and admin tools

You already know the app isn't doing the programming justice. This memo isn't about telling you what's broken — it's about what FBB could become with a foundation that matches the quality of the work Marcus puts into every block.

2.5★ App Store rating 88 ratings • "Love the workouts, hate this app."	12+ Public bug fixes shipped Status page, Sept 2025 – Apr 2026	0 Watch / Wear apps PDF fallback for offline use
---	---	---

Sources: App Store + Google Play public reviews; functional-bodybuilding.com/pages/status; user-submitted screenshots.

1 • Where the app is today

The current app is a white-label hybrid build on AppRabbit. It works, it's shipping fixes weekly, and the team's public status page is more transparent than most companies in the category. That transparency is exactly why the diagnosis below isn't news to you.

What's working

- The programming is the strongest in the category. Even one-star reviewers carve out praise for it.
- The Persist track architecture — PUMP LIFT, PUMP CONDITION, PERFORM, MINIMALIST, Hybrid Running, plus Workshops — is a genuine differentiator no competitor matches.
- The track quiz, equipment profiles, and Bridge Week deload structure show real product thinking.
- FBB ships fixes faster than peers and runs a rare public bug log. The community feels the effort.

What's holding it back

- Hybrid architecture has a ceiling. The pattern of bugs — “playing a video wipes rep scheme,” superset data loss, network errors saving sets, login fragmentation — is the signature of a webview wrapper, not a polish problem.
- No offline mode. The official guidance is a weekly PDF for athletes without reliable Wi-Fi. That's a workaround, not a product.
- No Apple Watch or Wear OS. The two largest watch ecosystems in fitness are unreachable on this stack.
- The set logger is multi-tap, the timer is undersized, and there's no previous-set overlay during logging — the three things every serious lifter notices in the first session.

The user voice

“I miss the browser version, which was simple and easy to see your training day in one scroll. There are entirely too many clicks just to see each movement. I dread using the app each training session.” — App Store, 9/3/2025

From the team's own status page (April 2026)

“Network issues leading to lag and errors saving data” listed as the current top engineering priority. This is an architectural symptom, not a feature gap.

2 • Why a bad app hurts a great program

Programming sells the first month. The app sells months two through twelve. Every churn audit in this category lands on the same cluster of reasons: the logger is slow, the timer is broken, the workout doesn't sync, the watch app is missing. None of those are programming problems — they're execution-layer problems, and they cap how many people get to experience what Marcus actually built.

The compounding cost

- **Acquisition tax.** A 2.5★ store rating is a paid-marketing penalty. CPI rises, conversion from ad click to install drops, and word-of-mouth referrals stall at the screenshot stage.
- **Retention tax.** Every “your data is gone” status note is a churn event. Hybrid bugs land disproportionately on power users — the exact cohort Persist is built for.
- **Brand tax.** “Great coach, bad app” becomes the dominant phrase in the category. It's reversible — but only with a foundation change, not another patch.
- **Roadmap tax.** Watch apps, Live Activities, App Intents, on-device AI, HealthKit autoregulation, Whoop integration — none of these ship on a webview. The next 24 months of category-defining features are gated by the platform.

What competitors are doing while you're patching

- Hevy and Strong have set a logger speed bar that coach-led apps now have to meet. Both are routinely chosen by Persist subscribers as a “side” logger — which means FBB is paying to acquire users who then split their trust with another app.
- TrainHeroic ships seven dedicated workout timers (EMOM, AMRAP, For Time, Tabata, Custom Interval, etc.). FBB programs need every one of them.
- Future and Centr have made Apple Watch the primary surface, not a companion. Athletes increasingly judge a fitness app by its watch face.
- MacroFactor has redefined nutrition tracking around adaptive macros and AI photo logging. The bar in nutrition has moved while FBB's nutrition surface stayed flat.

The honest framing

FBB doesn't have an app problem. FBB has a platform problem expressed as an app problem. AppRabbit was the right way to ship v1 fast. It is the wrong way to ship v2.

3 • What FBB could become

There is a clean path to becoming the most opinionated, most reliable coach-led training app on the market. The opportunity is unusually concrete because the gaps are well-known and the competitive set's execution layer is highly reverse-engineerable.

The shift, side by side

FBB today	FBB rebuilt
Webview app, multi-tap drill-in	Native iOS + Android, single-scroll session view
Slow logger, no previous-set overlay	Hevy-grade one-tap logger with prev-set values inline
Tiny built-in timer	Live Activities + Dynamic Island + watch haptic
Network-dependent set saves	Offline-first SQLite, syncs silently when Wi-Fi returns
No Apple Watch / Wear OS	Standalone watch app: lift with phone in the locker
Video can wipe log state	Pinned mini-player, log state survives every transition
Pre-listed alternates only	Equipment-aware Smart Replace with movement taxonomy
Manual progression	RPE-aware load suggestions; Bridge Week auto-detected
Facebook group + Atom forum	In-app per-track leaderboards, streaks, cheers, share cards
Basic macro logging	Barcode + AI photo + voice logging; adaptive macros
No recovery integration	Whoop / Oura / Apple HRV feed into autoregulation hints

The phased rollout

Each phase ships independent, user-visible value. Phase 1 alone fixes the brand wound and resets the App Store narrative.

Phase	Outcome	User-visible win
Phase 1 Native shell + logger parity	iOS + Android native, offline-first SQLite, PowerSync, Heavy-grade set logger, supersets, RPE, tempo, plate calculator, video that doesn't wipe state.	"It finally feels like a real app. My data doesn't disappear."
Phase 2 Watch + OS polish	Standalone Apple Watch + Wear OS. Live Activities, Dynamic Island, App Intents, widgets. EMOM/AMRAP/For Time/Custom Interval timers. Sanity CMS for programs.	"I can leave my phone in the locker and lift from my watch."
Phase 3 Smart programming + community	RPE-aware progression suggestions. Equipment-aware Smart Replace. Per-track leaderboards, streaks, cheers. Coach Form Review. Whoop/Oura/HRV-aware autoregulation.	"My training adapts to how I actually feel. The community keeps me showing up."
Phase 4 AI layer	On-device voice logging (WhisperKit). Pose-based rep counter and tempo cues for hero lifts. AI photo nutrition + adaptive macros. LLM next-workout briefer.	"This is the only app combining Marcus's programming with AI that actually helps."

4 • Why me, why now

I've gone deep on the current app, the AppRabbit platform, the Persist programming structure, the public bug log, and the competitive set. The teardown isn't speculation — it's sourced from your own status page, the App Store / Google Play review corpus, and side-by-side benchmarks against Hevy, Strong, TrainHeroic, Fitbod, Future, Centr, Caroline Girvan CGX, Caliber, and MacroFactor.

What I bring

- A fully-formed native architecture: Swift 6 / SwiftUI + Kotlin / Jetpack Compose, offline-first SQLite via PowerSync, Sanity CMS for programs and Movement Library, Bunny / Mux for video, RevenueCat for subscriptions.
- A backend and infrastructure plan FBB owns end-to-end: typed API, Postgres of record, content + user-data sync, observability and cost controls — designed so you can hire one engineer to maintain it, not a vendor to depend on.
- An internal admin console your coaches will actually use: program authoring with tempo, RPE, percent-based blocks; Movement Library management; user / subscription / form-review workflows; content scheduling for Bridge Weeks and Workshops.
- A workout execution flow designed for FBB's programming language — tempo, RPE, supersets, AMRAP back-offs, EMOM / E2MOM, percent-based Olympic blocks, Bridge Week deload detection — plus the modern OS surface (Apple Watch + Wear OS, Live Activities, App Intents, HealthKit) that's impossible on a webview.

Why I'm offering this pro bono

I'm a Persist subscriber. I want this app to be great because I use it. The fastest way I know to get there is to build it — not pitch it. So I'm not asking for a contract, a retainer, or equity. I'm asking for the access and trust to lead a full rebuild alongside your team — the iOS and Android apps, the backend and APIs they run on, the infrastructure they live in, and the internal admin tools your coaching and ops teams use — with the work shipping under the FBB name.

Doing the whole stack at once is the only way the offline-first sync, the coach authoring tools, and the player apps can be designed as one product. What I'd need from you: programming source-of-truth, data export from the current backend, decision-making access to the people who own coaching content, ops, and finance, and a single product owner for weekly sync. What you'd get: native iOS + Android apps, a clean backend the FBB team owns, and an admin console your coaches will actually want to use.

The bottom line

You've already built the hardest thing: a program people trust their bodies to. The app, the backend, and the tools your coaches use are the last mile, and right now they're the part of the experience users — and your own team — apologise for. A full native rebuild closes the gap between how good Persist is and how good using and running Persist feels. **That's the conversation I'd like to have on this call.**